Agentic AI System Design for Insider Trading Sentiment Analysis

Executive Summary

This report designs a production-ready, multi-agent AI backend that analyzes SEC Form 4 insider trades, overlays social sentiment from X (Twitter), stores knowledge using RAG-style indices, exposes a grounded chat interface, generates charts, and continuously improves via a closed learning loop using the Hermes Agent framework with OpenRouter-hosted LLMs. The analysis proposes three distinct architectures: (1) a classic supervised modular agent pipeline, (2) an event-driven asynchronous agent mesh, and (3) a hybrid PageIndex plus LightRAG memory-enhanced system.[1][2] [^3]

Existing open-source projects for insider trading analysis (such as `insider-trading-analyzer`) show effective patterns for parsing Form 4 into DuckDB and screening for high-signal insider trades, while SEC API clients like `sec-api-python` and MCP-based EDGAR servers power programmatic access to insider data. For retrieval, PageIndex demonstrates state-of-the-art reasoning-based RAG on FinanceBench using document trees instead of vectors, whereas LightRAG combines vector and knowledge-graph retrieval and offers production-grade storage backends like Neo4j, PostgreSQL, and Faiss. Real-world multi-agent financial assistants built with cloud providers (Amazon Bedrock), LangGraph, and Kafka-style streaming validate the use of multi-agent orchestration, event-driven pipelines, and observability in financial RAG systems.[4][5] [6][7] [8][9] [10][11] [12][13] [14][15] [16][17]

The recommended starting point for an internship-grade but production-aligned build is Architecture 1: a supervised modular Hermes agent pipeline backed by LightRAG for retrieval and a relational/analytic store for raw Form 4 and sentiment data. Architecture 2 (event-driven) is better for real-time, always-on operation with many tickers and higher volume, while Architecture 3 is ideal when deep reasoning over long financial documents is critical, taking advantage of PageIndex's tree-based retrieval for filings like 10-Ks and complex Form 4 footnotes.[5][10] [11][12] [17][1] [^4]

Problem Breakdown by System Layer

Data Layer

The system must ingest SEC Form 4 insider trading data for the last 24 hours, extract transaction details (insider, ticker, transaction type, share count, price), and compute the dollar value of each trade. Libraries such as `sec-api-python` (via `InsiderTradingApi`) and EDGAR-focused toolkits like `sec-edgar-agentkit` and `edgar-sec-mcp` already expose Form 4 search endpoints, providing JSON structures that can be transformed into normalized tables. For social sentiment, Apify X/Twitter scrapers provide tweet-level JSON with timestamps,

text, engagement stats, and basic user metadata, which can be filtered by ticker cashtags or issuer name over a seven-day window.[6][7] [8][18] [19][20] [15][21] [^22]

A practical storage approach is to follow patterns from `insider-trading-analyzer` and use DuckDB or PostgreSQL as the primary analytical store for Form 4 plus derived metrics, while using a document/RAG store such as LightRAG or PageIndex for unstructured text (raw filings and tweets). This separation allows efficient analytical queries for ranking and charting while keeping retrieval-optimized indices for question answering.[7][11] [^5]

Processing Layer

The processing layer is responsible for fetching, parsing, filtering, and enriching data. Existing guides on tracking Form 4 filings show how to query filings by ticker and date range and extract fields such as transaction code, price, and share counts, which can be adapted into a Form 4 ETL pipeline. Projects like `insider-trading-analyzer` demonstrate post-ingestion filters such as “high-value buys” and “cluster buys,” as well as backtesting and signal enrichment, providing a template for the Ranking and Enrichment modules.[6][7]

For X data, Apify’s Twitter scrapers support search queries and profile scrapes, returning tweet-level JSON including text, timestamps, and engagement metrics, which can be passed through a sentiment classifier. Processing also includes transforming tweets and filing text into the indexing format required by LightRAG (chunks plus entities) or PageIndex (PDF to tree structures), as described in their respective quickstart examples.[11][20] [21][22] [4][5]

Intelligence Layer

The intelligence layer combines retrieval, reasoning, and sentiment analysis. Standard RAG systems pair a vector database with embeddings and chunking, but often struggle on long financial documents where relevance depends on structure rather than surface similarity. PageIndex addresses this by transforming PDFs into hierarchical tree structures with semantic node summaries and page references, enabling reasoning-based retrieval that achieved about 98.7% accuracy on FinanceBench for financial document QA.[10][13] [14][17] [^4]

LightRAG implements a hybrid of vector retrieval and knowledge-graph reasoning, supporting local, global, hybrid, naive, and “mix” modes and integrating with multiple storage engines (Neo4j, PostgreSQL, Faiss, Oracle) for scalable production deployments. Hermes provides the agent loop, skill system, and closed learning capabilities (memory, skill creation, and self-improvement) while delegating subagents for parallel tasks and integrating with LLM providers such as OpenRouter. Sentiment can be implemented either using an LLM-based classifier over tweet text or a lightweight sentiment model, but in all cases results should be stored and surfaced through the RAG layer to keep responses grounded.[3][1] [5][11]

Interface Layer

The user interface is a chat-style API or CLI that routes messages through Hermes, which supports a terminal UI and messaging gateways (Telegram, Discord, Slack, etc.) on top of a single backend agent process. Hermes' tools and skills systems allow defining custom tools for querying the database, interacting with LightRAG/PageIndex, calling Apify, and generating charts, which are then invoked by the Chat Agent according to tool-calling directives.[1][5] [^11]

Visualizations can be produced by a dedicated Visualization Agent that uses Python plotting libraries (e.g., Matplotlib or Plotly) to generate images charting insider trade dollar values, sentiment distributions, and timeline overlays, similar to dashboards proposed as future add-ons in `insider-trading-analyzer` and other trading tools. The interface must enforce grounded responses: the Chat Agent is instructed to answer strictly from retrieved context and to respond with explicit uncertainty when data is missing.[2][3][7]

Learning Layer

The learning layer must implement a closed loop where the system inspects its own traces and user interactions to improve future behavior. Hermes natively supports a "closed learning loop" with agent-curated memory, autonomous skill creation, and periodic nudges to persist useful knowledge, plus tools for scheduled audits and trajectory generation for RL-style improvement. This can be harnessed by defining learning-focused skills that review logs (e.g., questions with low retrieval scores or user corrections) and adjust prompts, retrieval parameters, or ranking thresholds.[3][1]

Real-world agentic RAG systems integrate observability and evaluation to monitor retrieval quality and answer accuracy, for example by using LangGraph plus Qdrant and Weave in a financial agentic RAG pipeline, or by combining agent collaboration with knowledge bases and metrics in Amazon Bedrock. These patterns can be mirrored: a Learning Loop Agent periodically samples past queries, re-evaluates them, and updates Hermes skills and configuration, forming a practical closed learning feedback loop.[12][13]

Technology Evaluation

SEC / Insider Trading Data Access

1. `sec-api-python` (InsiderTradingApi)
   - Provides a high-level API to query insider trades by ticker and other criteria, returning JSON structures that include transactions, symbols, and filing timestamps, as demonstrated in its `InsiderTradingApi` usage examples.[^8]
   - Pros: Managed API with rich coverage across many SEC forms and robust indexes; fast to integrate; no need to handle EDGAR scraping and rate limits directly.[^8]
   - Cons: Commercial API with key management and usage limits; introduces vendor dependency.
2. EDGAR MCP and Agent Kits (`edgar-sec-mcp`, `sec-edgar-agentkit`)

- `edgar-sec-mcp` exposes tools such as `GetForm4ByTicker` via the Model Context Protocol, designed explicitly for AI agents.[^15]
- `sec-edgar-agentkit` builds on this MCP server to integrate EDGAR access into multiple agent frameworks including LangChain, Gradio, Dify, and smolagents, providing a multi-framework monorepo for EDGAR-driven agents.[^19]
- Pros: Natively agent-friendly, integrates with tooling ecosystems, and keeps data source logic modular and replaceable.[19][15]
- Cons: Requires running and maintaining the MCP server; some ETL logic still has to be implemented.

3. **Direct EDGAR Download with Custom ETL (`insider-trading-analyzer`)**
   - Downloads Form 4 and Form 144 filings, parses them into DuckDB, and applies filters for high-value and cluster insider buys, while also enabling backtesting against price data.[^7]
   - Pros: Full control over data pipeline, local-first storage, and flexibility to augment with additional signals such as earnings, short interest, and news.[^7]
   - Cons: More engineering effort; must manage SEC crawling, EDGAR HTML parsing, and schema evolution.

For an internship-scale, agentic system that still feels realistic, using `sec-api-python` or EDGAR MCP as the primary ingestion mechanism is pragmatic, while reusing ideas from `insider-trading-analyzer` for schema design and ranking logic.[15][8] [19][7]

Social Sentiment via Apify

Apify provides multiple Twitter/X scrapers that expose programmatic APIs to collect tweets by search query, profile, or hashtag, returning JSON objects with text, timestamps, engagement metrics, and basic user metadata. Example scripts show how to supply an API token and a query (e.g., `from:elonmusk` or keyword-based queries) and retrieve paginated results, which can be consumed by Python code via HTTP requests.[18][20] [21][22]

Compared to building a custom scraper or paying for the official X API, Apify's hosted scrapers address challenges like rotating proxies, rate limits, and API changes, making it suitable for a robust, low-maintenance sentiment ingestion agent. The trade-off is vendor dependency and per-run cost, so the architecture should abstract the Social Scraper Agent behind a skill tool, making it possible to swap in another provider later.[20][22] [^18]

Retrieval and Knowledge Store: RAG vs PageIndex vs LightRAG

1. **Traditional Vector RAG**
   - Uses embeddings and chunking stored in a vector database, then retrieves top-k
     chunks based on similarity and passes them to an LLM for generation.[9][13]
   - Pros: Well-understood, many libraries and managed services; strong fit when answers live in a small number of semantically similar chunks.[^13]

- Cons: Struggles with financial documents where relevant information may appear in tables, appendices, or sections with low semantic similarity to the query; can miss cross-references like "see Table 5.3."[17][4] [^10]

2. **PageIndex (Reasoning-Based RAG)**
   - Converts PDFs into hierarchical tree structures (nodes with titles, summaries, and page ranges) and uses reasoning-based retrieval rather than semantic similarity, eliminating the need for vector databases and arbitrary chunking. It has been used to build a financial RAG system achieving approximately 98.7% accuracy on FinanceBench, outperforming vector-based RAG on professional document QA.[4][10] [^17]
   - Pros: Strong performance on long, structured financial documents; explicit page references and tree-structured reasoning; avoids managing vectors and chunk parameters. Ideal when queries require understanding of document structure, section relationships, and cross-references.[14][10] [^4]
   - Cons: Currently optimized for GPT-style models; better suited for long PDFs (10-Ks, long reports) than for short, dynamic text like tweets; adding new documents requires rerunning tree generation.[10][4]
3. **LightRAG (Hybrid Vector + Graph RAG)**
   - Builds a knowledge graph plus vector index, supporting retrieval modes such as `local` (entity-focused), `global` (relationship-focused), `hybrid`, `naive`, and `mix` (combining KG and vector retrieval). It integrates with storage options including NetworkX, Neo4j, PostgreSQL (with pgvector and Apache AGE), Faiss, and Oracle, and can run in-process or as a FastAPI server.[5][11]
   - Pros: High flexibility across retrieval strategies; strong support for multi-document, multi-entity knowledge graphs; production-friendly storage backends and performance optimizations; suitable for both financial filings and social data. Supports custom KGs and editing entities/relations, making it well suited for modeling insiders, tickers, and sentiment relationships.[11][5]
   - Cons: More moving parts and configuration (KG, vector DB, storage); requires careful tuning of chunk sizes and thresholds; somewhat heavier than a minimal vector-only RAG.[5][11]

Given the project's mix of moderately sized SEC filings (Form 4) and large volumes of short tweets, LightRAG is a strong default knowledge store: it handles text and graph structure, scales with PostgreSQL or Neo4j, and supports multi-mode retrieval for flexible QA. PageIndex is most beneficial when the scope is expanded to long-form filings (e.g., 10-Ks, 10-Qs, or large proxy statements), where its FinanceBench results indicate clear advantages over vector RAG; thus it is primarily recommended in the hybrid Architecture 3.[17][4] [10][11] [^5]

Agent Orchestration and Learning: Hermes

Hermes is a self-improving AI agent framework in Python that provides a terminal-based and messaging-based interface, supports multiple model providers (including OpenRouter), and implements a closed learning loop with agent-curated memory, skills that self-improve during use, and autonomous skill creation. It can spawn isolated subagents for parallel workstreams and call tools via RPC, reducing prompt context overhead while still coordinating complex pipelines.[1][3]

Hermes' skills and tools system is compatible with the agentskills.io open standard and integrates with MCP servers, making it suitable for plugging in EDGAR MCP, RAG backends, and visualization tools as agent tools. Its built-in scheduler (cron) enables daily reports and unattended workflows, which is ideal for running periodic Form 4 ingestion and analytics jobs, while its learning loop facilities provide a natural hook for the Learning Layer.[19][15] [^1]

LLM Provider: OpenRouter

OpenRouter exposes a large catalog of models from multiple vendors and includes a number of models with free tiers or entirely free usage, such as Mistral and LLaMA 3-based instruction-tuned models. Documentation and curated lists of free models show options like `mistralai/mixtral-8x7b-instruct`, `meta-llama/llama-3-8b-instruct`, and the `openrouter/auto` meta-endpoint, which automatically chooses an appropriate free model.[2][24]

For this project, using `openrouter/auto` for general chat and a smaller, faster model for classification tasks (e.g., tweet sentiment) is reasonable, while keeping the provider abstracted through Hermes' model configuration so that models can be swapped without code changes.[2][1]

Data Pipeline and Multi-Agent Patterns in Finance

Industry articles and case studies demonstrate that multi-agent RAG systems in finance benefit from specialized agents for retrieval, data transformation, and analysis, orchestrated either by a supervisor agent or via an agent graph. A WandB field guide, for example, describes a financial agentic RAG pipeline using Qdrant for retrieval, LangGraph for multi-agent orchestration, and Weave for observability, emphasizing routing queries to appropriate tools and maintaining evaluation metrics.[9][12] [^13]

Similarly, AWS Bedrock's multi-agent investment assistant uses supervisor and subagents to analyze news, evaluate stocks, and optimize portfolios, combining retrieval from knowledge bases with specialized analytic tools. Financial institutions have combined agentic AI, RAG, and data streaming (Kafka) for real-time decisioning in regulated FinTech environments, confirming the value of event-driven architectures and streaming topics as the backbone for multi-agent collaboration.[16][12]

Architecture 1: Supervised Modular Hermes Agent Pipeline

High-Level Diagram (Textual)

- User / Client → Hermes Chat Gateway → **Supervisor Agent** → orchestrates:
  - **SEC Data Agent** (Form 4 ingestion)
  - **Ranking Agent** (top trades by dollar value)
  - **Social Scraper Agent** (Apify X/Twitter)
  - **Sentiment Agent** (tweet classification)
  - **Indexing Agent** (LightRAG / RAG store)
  - **Retrieval Agent** (query-time context assembly)
  - **Chat Agent** (grounded response generation)
  - **Visualization Agent** (chart generation)
  - **Learning Loop Agent** (closed-loop optimization)

Data storage consists of:

- Relational/analytical DB (DuckDB/PostgreSQL) for structured Form 4 and sentiment aggregates.
- LightRAG store for unstructured text and knowledge graph relationships.

Agent Roles and Responsibilities

- **SEC Data Agent**
  - Periodically (e.g., every 15–60 minutes) queries insider-trading endpoints using `InsiderTradingApi` or EDGAR MCP tools, filtered to trades filed in the last 24 hours.[6][8][^15]
  - Parses JSON into normalized tables (`filings`, `transactions`, `insiders`) following patterns from `insider-trading-analyzer` and Form 4 guides.[6][7]
- **Ranking Agent**
  - Computes trade dollar value as price × shares and ranks by total value within the last 24 hours, filtering trades to instruments of interest (e.g., common stock, purchases only).
  - Selects the Top 5 trades and writes a `top_trades` table as well as emitting events or messages for downstream agents.
- **Social Scraper Agent**
  - For each symbol in `top_trades`, constructs Apify queries such as `"$TICKER OR company_name"` and fetches tweets from the last 7 days, handling pagination and rate limits.[22][18][^20]
  - Stores tweet JSON in a `tweets` table, including tweet ID, text, timestamp, user metadata, and engagement metrics.

- Sentiment Agent
  - Reads new tweets and applies a sentiment classifier (LLM via OpenRouter or a local model), producing labels such as `bullish`, `bearish`, and `neutral` (or a continuous sentiment score).
  - Aggregates sentiment by ticker and time bucket (e.g., daily sentiment index) and writes to `sentiment_summary`.
- Indexing Agent
  - Converts Form 4 filing text (and optionally relevant 10-K/10-Q sections) plus tweets into LightRAG's ingestion format and calls `rag.insert()` or corresponding API functions.[11][5]
  - Configures chunking parameters (`chunk_token_size` around 600–800 for filings, smaller for tweets) and ensures incremental insertion as new data arrives.[^11]
- Retrieval Agent
  - Given a user question, determines which tickers and time windows are relevant and queries LightRAG with the appropriate mode (e.g., `local` for entity-focused questions like "What did insiders do on TSLA yesterday?" and `mix` for combining filings and sentiments).[5][11]
  - Pulls additional numerical context from the relational DB (e.g., trade values, sentiment aggregates) to feed into the Chat Agent.
- Chat Agent
  - Invokes the main LLM via OpenRouter and is instructed via system prompt to answer strictly from provided context snippets and numeric tables, with explicit instructions to answer "I don't know" when information is missing.
  - Formats citations (e.g., quoting filing IDs, dates, user handles) and ensures answers are structured and explainable.
- Visualization Agent
  - When a query requests charts ("plot sentiment vs insider buys for NVDA"), generates scriptable plotting commands (Matplotlib/Plotly) via a tool that creates PNG charts and returns references to the frontend.[^7]
  - Uses DB queries to retrieve time series, e.g., daily sum of insider trade value and average sentiment per ticker, and plots them with proper labeling.
- Learning Loop Agent
  - Periodically reviews logs and conversation history using Hermes' memory and skills, identifying:
    - Queries that returned low-confidence answers or frequent "I don't know."
    - Cases where users corrected or disagreed with outputs.
  - Updates prompts, agent routing heuristics (e.g., when to use `mix` vs `local` mode in LightRAG), and ranking thresholds for high-signal trades, leveraging Hermes' self-improving skills and scheduled automations.[^1]

Data Flow

1. Cron-scheduled SEC Data Agent fetches Form 4 filings (last 24 hours), normalizes them into the DB, and marks new transactions.
2. Ranking Agent computes trade dollar values, selects Top 5, and writes to `top_trades`.
3. Social Scraper Agent fetches tweets for each top-trade ticker and stores them.
4. Sentiment Agent classifies tweets and writes both per-tweet labels and aggregated sentiment summaries.
5. Indexing Agent ingests filings and tweets into LightRAG and updates the knowledge graph.
6. User queries arrive through Hermes; Supervisor Agent delegates to Retrieval Agent, which queries LightRAG and DB, and returns a context bundle.
7. Chat Agent formulates grounded responses and optionally triggers Visualization Agent to generate charts.
8. Learning Loop Agent periodically inspects logs, updates skills and prompts, and may trigger backfill jobs (e.g., indexing additional filings for popular tickers).

Pros, Cons, and Scalability

- Pros
  - Clear separation of responsibilities among agents; easier for an intern to build and test each agent as an independent Hermes skill.
  - LightRAG provides flexible retrieval across text and graph structure and production-ready storage backends, supporting growth in data volume and complexity.[5][11]
  - Hermes' learning loop and scheduler enable continuous improvement and unattended operation.[^1]
- Cons
  - Supervisor-centric orchestration can become a bottleneck if all requests funnel through a single agent; requires careful design of tools to keep context small.
  - The pipeline is primarily batch-oriented; near-real-time reaction to new filings requires more frequent cron intervals and tuning.
- Scalability
  - Horizontal scaling of SEC Data, Social Scraper, and Sentiment Agents is possible by running multiple Hermes instances or worker processes with partitioned ticker sets.
  - LightRAG can scale via external databases (PostgreSQL, Neo4j, Faiss) and by tuning chunking and thresholds; retrieval remains efficient even with many documents.[11][5]
  - For very high query volume, Chat Agents can be replicated behind a load balancer, as Hermes supports running on VPS or serverless infrastructure.[^1]
- Real-World Inspiration

- Mirrors AWS Bedrock's multi-agent financial assistant (supervisor plus subagents), the WandB financial agentic RAG pipeline using specialized retrieval and orchestration, and `insider-trading-analyzer`'s pipeline for signal-focused Form 4 analytics.[12][13] [^7]

Architecture 2: Event-Driven Asynchronous Agent Mesh

High-Level Diagram (Textual)

- **Event Bus (Kafka / Redis Streams / SQS)** at the center.
- Producers:
  - SEC Ingestion Agent → emits `Form4Ingested` events.
  - Ranking Agent → emits `TopTradesUpdated` events.
  - Social Scraper Agent → emits `TweetsIngested` events.
  - Sentiment Agent → emits `SentimentUpdated` events.
- Consumers:
  - Indexing Agent → consumes data events and updates RAG store.
  - Learning Loop Agent → consumes evaluation and error events.
  - Chat Orchestrator Agent → consumes query events and requests retrieval.

Hermes agents are deployed as microservices (or processes) that subscribe to topics and coordinate via the event bus rather than through a single supervisor.

Agent Roles and Responsibilities

Agent roles are broadly similar to Architecture 1 but decoupled via events:

- **SEC Ingestion Agent**
  - Continuously polls `InsiderTradingApi` or EDGAR MCP for new Form 4 filings and publishes each parsed filing as a `Form4Ingested` event.[8][15]
- **Ranking Agent**
  - Subscribes to `Form4Ingested`, maintains rolling views of trade values in the database, and periodically emits `TopTradesUpdated` events with the current Top 5 trades.
- **Social Scraper Agent**
  - Subscribes to `TopTradesUpdated` and triggers Apify runs for affected tickers, publishing `TweetsIngested` events as tweet batches arrive.[20][22]
- **Sentiment Agent**
  - Subscribes to `TweetsIngested`, performs sentiment analysis, and emits `SentimentUpdated` events with per-ticker summaries.
- **Indexing Agent**

    - Subscribes to all three event types, updating LightRAG or PageIndex indices incrementally.
- Chat Orchestrator Agent
    - For incoming user queries, publishes a `QueryRequested` event; a Retrieval Agent consumes this, gathers context, and publishes `QueryContextReady`, which is consumed by a Response Agent that produces the answer and any charts.
- Learning Loop Agent
    - Subscribes to `QueryAnswered`, `Error`, and `LowConfidence` topics for evaluation and improvement.

Data Flow

1. SEC Ingestion Agent streams new Form 4 filings and publishes `Form4Ingested` events.
2. Ranking Agent consumes these events, recomputes Top 5 trades, and publishes `TopTradesUpdated`.
3. Social Scraper Agent listens for `TopTradesUpdated` and triggers Apify scraping for each ticker; as results arrive, it publishes `TweetsIngested`.
4. Sentiment Agent consumes `TweetsIngested`, classifies sentiment, stores results, and emits `SentimentUpdated` events.
5. Indexing Agent consumes all data events and updates RAG indices in near real time.
6. When a user asks a question, Chat Orchestrator publishes `QueryRequested`; Retrieval Agent assembles context using the latest DB and RAG state, then publishes `QueryContextReady`.
7. Response Agent generates an answer and charts, then publishes `QueryAnswered` and writes results to logs.
8. Learning Loop Agent monitors all relevant topics and triggers prompt or configuration updates as needed.

Pros, Cons, and Scalability

- Pros
    - Strong decoupling between agents; teams can independently deploy and scale ingestion, sentiment, indexing, and chat components.
    - Naturally supports near-real-time updates and many tickers, aligning with real-world event-driven FinTech architectures that integrate Kafka with agentic AI and RAG. [^16]
    - Failures are isolated: if Apify scraping fails, other agents continue operating; events can be retried or dead-lettered.
- Cons
    - Higher infrastructure complexity (event bus, monitoring topics, handling idempotency and ordering).

    - Harder to reason about end-to-end flows for interns compared to the more linear pipeline of Architecture 1.
- Scalability
    - Each agent type scales independently by adding more consumers to its topics (e.g., multiple Sentiment Agents for heavy tweet throughput).
    - RAG backends (LightRAG/PageIndex) can be served via FastAPI and scaled behind load balancers. Event backpressure can be handled via partitioning and consumer groups.[^5]
- Real-World Inspiration
    - Directly inspired by regulated FinTech deployments that use event-driven architectures (Kafka) plus agentic AI and RAG on Vertex AI, and by general industry practice for streaming financial data processing.[^16]

Architecture 3: Hybrid PageIndex + LightRAG Memory-Enhanced System

High-Level Diagram (Textual)

- Data stores:
    - PageIndex trees for long-form financial PDFs (10-Ks, proxy statements, detailed Form 4 attachments).
    - LightRAG store for tweets, short filings, and a knowledge graph of insiders, tickers, and sentiment.
    - Analytical DB for numeric aggregates.
- Agents (under Hermes or mixed orchestration):
    - Document Ingestion Agent (PageIndex)
    - Short-Text Ingestion Agent (LightRAG)
    - Hybrid Retrieval Agent
    - Chat Agent
    - Visualization Agent
    - Learning Loop Agent

Agent Roles and Responsibilities

- Document Ingestion Agent
    - Uses PageIndex to process long PDFs into tree-structured indices with node summaries and page ranges, following the provided run script and configuration (model selection, max pages per node, etc.).[^4]
    - Stores resulting JSON trees in a document index service or database, keyed by document ID and ticker.[^4]
- Short-Text Ingestion Agent

    - Handles Form 4 body text (when relatively short) and tweets, inserting them into LightRAG with appropriate chunk sizes and entity extraction, building a knowledge graph of insiders, issuers, and sentiment.[11][5]
- **Hybrid Retrieval Agent**
    - Implements retrieval strategy selection:
        - If the query is about granular document content (e.g., "What are the vesting conditions described in the latest equity award for AAPL insiders?"), it uses PageIndex to select nodes likely to contain the answer and pulls their full text.[10][4]
        - If the query is about relationships or high-level patterns (e.g., "Summarize insider buys and social sentiment for TSLA in the last week"), it uses LightRAG's `mix` or `global` modes plus DB queries.[5][11]
    - Merges the contexts from both systems into a unified bundle for the Chat Agent.
- **Chat Agent**
    - Similar to prior architectures but with prompts explicitly differentiating between PageIndex-derived context (with page references) and LightRAG-derived context (graph-based and tweet-based), allowing the answer to reference both "filing sections" and "social chatter."
- **Visualization Agent**
    - Generates charts that may overlay structured data (trade values, sentiment indices) with PageIndex-derived references (e.g., highlighting when a new compensation scheme was disclosed in a 10-K vs insider trades around that date).[4][7]
- **Learning Loop Agent**
    - Monitors which retrieval strategy leads to better user satisfaction (e.g., via explicit feedback or implicit metrics like follow-up questions), adjusting the criteria used by the Hybrid Retrieval Agent to choose between PageIndex and LightRAG.

Data Flow

1. Long financial documents are downloaded (10-Ks, detailed Form 4 attachments) and processed by PageIndex into trees, which are stored by ticker and filing type.[^4]
2. Form 4 summaries and tweets are ingested into LightRAG, which maintains a knowledge graph and vector index.[11][5]
3. User queries arrive; Hybrid Retrieval Agent classifies them into "document-heavy" vs "pattern/sentiment-heavy," then pulls context from PageIndex, LightRAG, and the DB as appropriate.
4. Chat Agent answers, grounded in the combined context.
5. Visualization Agent optionally creates charts linking quantitative signals with document events.
6. Learning Loop Agent regularly reviews logs to tune routing rules and retrieval parameters.

Pros, Cons, and Scalability

- Pros
  - Best retrieval quality for complex financial questions that span structured filings and market sentiment; leverages PageIndex's strong performance on FinanceBench and LightRAG's flexible multi-source retrieval.[17][10] [4][5] [^11]
  - Highly extensible; adding support for other document types (e.g., 13F holdings, earnings call transcripts) is natural.
- Cons
  - More complex retrieval logic and indexing pipelines; requires careful design of query classification and context merging.
  - Higher compute cost due to running both PageIndex and LightRAG, plus possible duplication of data.
- Scalability
  - PageIndex processing is heavier but primarily offline/batch; LightRAG and the DB handle online queries and incremental updates.
  - Both PageIndex and LightRAG can be deployed as services and scaled independently, while Hermes agents coordinate their usage.[4][5]
- Real-World Inspiration
  - Aligns with emerging "reasoning-based RAG" patterns that combine structured indexing of documents (like PageIndex) with standard RAG for broader knowledge coverage, particularly in finance where document structure and cross-references matter.[14][10] [17][4]

Optimization and Risk Analysis

Bottlenecks and Latency

- SEC Data Latency and Rate Limits
  - `sec-api-python` and EDGAR endpoints impose rate and usage limits; ignoring these can cause delays or dropped data. Batch querying and caching results in the DB, plus backoff and retry logic in the SEC Data Agent, are essential.[^8]
- Apify Scraper Latency
  - Apify runs may take seconds to minutes and incur per-run costs. To reduce user-facing latency, Social Scraper Agents should pre-fill tweet caches for popular tickers and run asynchronously, while query-time sentiment relies on cached results.[18][20]
- LLM Latency and Cost
  - Tweet-level sentiment classification and answer generation both incur LLM calls; using smaller, faster models for classification and sharing context across classification batches reduces overhead.[24][2]
- Indexing Overhead

- LightRAG's entity extraction and graph building, and PageIndex's PDF processing, are CPU and token intensive. Scheduling indexing during lower-traffic periods and using batch insertion (as supported by LightRAG) help manage cost and latency.[4][5][^11]

Data Quality and Leakage Risks

- Data Quality
  - SEC filings may include amended or corrected Form 4s; the ETL should handle duplicates and corrections, potentially by tracking accession numbers and amendment flags.[^6]
  - Tweet text includes spam, bots, and sarcasm; naive sentiment analysis may misinterpret these, so sentiment models and thresholds must be tuned and monitored.
- Data Leakage and Compliance
  - Tweets include usernames and may include personal data; storing and processing this data, especially if combined with other datasets, raises privacy considerations and may require appropriate anonymization or limited retention.
  - API keys for SEC data, Apify, and OpenRouter must be managed securely, avoiding leaks in logs or user-visible responses and following least-privilege practices.

Monitoring and Observability

- Metric Collection
  - Capture metrics such as ingestion lag (time from filing to ingestion), tweet coverage per ticker, sentiment classification distribution, RAG retrieval scores, and LLM latency.
  - Observability setups in multi-agent financial RAG pipelines typically track retrieval quality and tool usage; similar metrics can be integrated using Hermes logs plus external dashboards.[13][12]
- Evaluation Loops
  - Sample answers should be regularly re-evaluated using offline judges or LLM-based evaluators to detect hallucinations and retrieval failures, following practices described in agentic RAG evaluation guides.[^13]
- Error Handling
  - Agents should implement structured error types (e.g., upstream API failure, parsing error, DB write failure) and publish them to a central error topic or log for the Learning Loop Agent to act on.

Hallucination Mitigation

- Strictly constrain the Chat Agent with prompts that enforce answering from context only, and instruct it to respond with "I don't know" when necessary.
- Provide explicit citations in responses (e.g., filing dates, insider names) pulled directly from the DB/RAG context so that users can verify statements.
- Use retrieval confidence thresholds (e.g., top-k similarity scores, node selection rationale in PageIndex prompts) and require minimum evidence before certain claims are made.[10] [5] [^4]

Final Recommendation

For an AI internship assignment that must be realistic yet implementable, **Architecture 1 (Supervised Modular Hermes Agent Pipeline)** is the best primary choice. It balances clarity and modularity with production-aligned patterns, leveraging Hermes' learning loop and tools, LightRAG's hybrid retrieval and scalable storage backends, and well-documented SEC and Apify integrations. Each agent can be implemented as a discrete Python module and Hermes skill, with well-defined inputs and outputs, making it ideal for incremental development and testing.[18][20] [7][8] [6][1] [5][11]

Architecture 2 (Event-Driven Agent Mesh) is preferable when targeting high-throughput, near-real-time scenarios with many tickers and a requirement for robust failure isolation and independent scaling, closely mirroring event-driven FinTech deployments that combine Kafka, RAG, and agentic AI. However, its infrastructure complexity makes it less suitable as a first internship deliverable, though it can be framed as a stretch or "Phase 2" goal.[12][16]

Architecture 3 (Hybrid PageIndex + LightRAG) is recommended when the main value proposition shifts toward deep analysis of complex financial documents (earnings reports, 10-Ks, detailed Form 4 narratives) combined with sentiment overlays, leveraging PageIndex's state-of-the-art reasoning-based RAG capability on FinanceBench and LightRAG's flexible graph retrieval. It is an excellent direction for research-oriented extensions or when working with institutions whose workflows revolve around long-form disclosures.[14][17] [10][5] [11][4]

Taken together, these architectures provide a roadmap: start with Architecture 1 for a solid, agentic MVP; evolve toward Architecture 2 for real-time robustness; and integrate Architecture 3's hybrid retrieval to maximize answer quality on complex financial questions.

References

1. README.md - NousResearch/hermes-agent - The agent that grows with you. Contribute to NousResearch/hermes-agent development by creating an ac...
2. List of Free Models on OpenRouter - SuperSharpAI
3. Hermes Agent is open source and built in Python, so it's easy for ... - X
4. PageIndex/README.md at main · VectifyAI/PageIndex - PageIndex: Document Index System for Reasoning-based RAG - VectifyAI/PageIndex

5. LightRAG-hku/lightrag/api/README.md at main - LightRAG requires both LLM and Embedding Model to work together to complete document indexing and qu...
6. Tracking Insider Trading with Form 4 - A comprehensive guide to analyzing insider trading activity using Form 4 filings with edgartools
7. wescules/insider-trading-analyzer: A Python project that ... - Features · Parses and stores insider filings into DuckDB · Screens for high-value and cluster inside...
8. sec-api-python/examples.py at master - Python SEC EDGAR Filings API. Over 18 million filings, all 150 filing types supported. Query, full-t...
9. What is a RAG AI agent? - RAG AI agents are built to act on behalf of users by combining retrieval, reasoning, and action. Her...
10. PageIndex: Vectorless RAG with Reasoning-Based Document ... - PageIndex is an open-source RAG framework that eliminates vector embeddings and chunking, replacing ...
11. LarFii/LightRAG-hku: "LightRAG: Simple and Fast Retrieval ... - Video demo of running LightRAG locally. All the code can be found in the examples . Set OpenAI API k...
12. Part 3: Building an AI-powered assistant for investment research ... - In this post, we walk through how to build a multi-agent investment research assistant using the mul...
13. Building a financial agentic RAG pipeline (Part 1) - A comprehensive guide to optimized retrieval with Qdrant, agentic RAG orchestration via LangGraph, a...
14. PageIndex outperforms vector-based RAG systems in finance - Here's the 60-second mental model for RAG vs GraphRAG vs LightRAG: RAG (vector RAG): • Best when the...
15. README.md - flothjl/edgar-sec-mcp - edgar-sec-mcp is an MCP server designed to facilitate AI agents in accessing and interacting with ED...
16. Agentic AI and RAG in Regulated FinTech with Apache ... - This blog post explores how regulated FinTechs are succeeding with event-driven architecture and rea...
17. New RAG Approach Achieves 98.7% Accuracy on FinanceBench - Researchers introduced a new RAG approach that achieves 98.7% accuracy (SOTA) on a financial benchma...
18. Mastering Twitter Data Collection: A Comprehensive Guide to Efficient Scraping Solutions - Introduction Twitter data is gold for developers, researchers, and businesses. Whether...
19. stefanoamorelli/sec-edgar-agentkit: AI agent toolkit ... - A multi-framework monorepo toolkit for building AI agents and applications that can access and analy...
20. Twitter (X.com) Tweets & Profiles Scraper - Scrape tweets & profiles from any Twitter (X.com) account. Search, extract tweets with replies, or s...
21. Twitter (X) Scraper - Scrape tweets from any Twitter profile — hashtags, threads, replies, media, followers, stats, and fu...
22. Twitter (X.com) Scraper Unlimited: No Limits - Example Cost Calculations · Basic Profile Scraping : Scraping one profile with query from:NASA; First...
23. Page 25 | Scripts Search Results for "track" - With regime-based trend track ing, customizable visuals, and a detailed status table, it supports bo...

24. Best Free Ai Models Openrouter 2026 - 24 completely FREE models on OpenRouter – no credit card needed. MiMo beats Claude on coding. Gemini...